

Relatório Técnico: Gerenciador de Livros CRUD

Aluno: Pedro Roberto Ribeiro Bandeira Labre

Matrícula: 202410630002

Índice

1. Introdução

- 1.1. Especificações Técnicas

2. Processo de Desenvolvimento

- 2.1. Configuração do Ambiente
- 2.2. Modelagem do Banco de Dados
- 2.3. Camada de Conexão ([conexao.php](#))

3. CREATE - Adicionar Livros

- 3.1. Fluxo de Funcionamento
- 3.2. Código Final com Prepared Statements
- 3.3. Formulário de Entrada ([templates/form_adicionar.php](#))

4. READ - Listagem de Livros

- 4.1. Lógica de Separação
- 4.2. Implementação do [index.php](#)
- 4.3. Renderização Dinâmica em [lista_livros.php](#)
- 4.4. Segurança

5. UPDATE - Edição de Livros

- 5.1. Funcionamento Dual
- 5.2. Lógica de Busca (GET)
- 5.3. Lógica de Atualização (POST)
- 5.4. Formulário de Edição ([templates/form_editar.php](#))

6. DELETE - Exclusão de Livros

- 6.1. Fluxo com Confirmação
- 6.2. Implementação do [excluir.php](#)
- 6.3. Interface de Confirmação ([templates/confirmar_exclusao.php](#))

7. Funcionalidade Extra: Troca Rápida de Status

- 7.1. Ciclo de Status
- 7.2. Implementação

8. Camada de Apresentação

- 8.1. Sistema de Templates
- 8.2. Arquivo: `templates/header.php`
- 8.3. Arquivo: `templates/footer.php`
- 8.4. Vantagens da Abordagem

9. Arquivo: `css/style.css`

- 9.1. Design "Neumorphic/Solid"
- 9.2. Sistema de Cores Consistente
- 9.3. Grid Responsivo
- 9.4. Tabela Estilizada
- 9.5. Responsividade

10. Desafios e Soluções: Conflito de Porta MySQL

- 10.1. Problema
- 10.2. Solução Implementada
- 10.3. Reconfiguração do XAMPP
- 10.4. Lições Aprendidas

11. Desafios e Soluções: Erro de Autenticação MySQL

- 11.1. Problema
- 11.2. Tentativas Iniciais (Sem Sucesso)
- 11.3. Solução Final
- 11.4. Descoberta Importante
- 11.5. Impacto

12. Desafios e Soluções: Corrupção da Tabela do Banco de Dados

- 12.1. Problema
- 12.2. Diagnóstico
- 12.3. Solução

13. Recursos e Ferramentas: XAMPP

- 13.1. Componentes Utilizados
- 13.2. Vantagens
- 13.3. Uso no Projeto

14. Recursos e Ferramentas: Visual Studio Code

- 14.1. PHP Intelephense
- 14.2. MySQL (Weijan Chen)
- 14.3. HTML CSS Support
- 14.4. Prettier - Code Formatter

15. Recursos e Ferramentas: Bootstrap Classes (Inline)

- 15.1. Decisão de Design
- 15.2. Classes Utilizadas
- 15.3. Justificativa

16. Recursos e Ferramentas: mysqli (PHP Extension)

- 16.1. Escolha Tecnológica
- 16.2. Vantagens do mysqli
- 16.3. Padrão de Uso

17. Recursos e Ferramentas: Chrome DevTools

- 17.1. Console JavaScript
- 17.2. Network Tab
- 17.3. Elements/Inspector

18. Recursos Consultados

- 18.1. php.net
- 18.2. dev.mysql.com
- 18.3. MDN Web Docs
- 18.4. Stack Overflow
- 18.5. W3Schools
- 18.6. HTML Color Codes

19. Técnicas Implementadas

- 19.1. Exibição de Erros

1. Introdução

Documentação do processo completo de desenvolvimento de um sistema CRUD (Create, Read, Update, Delete) para gerenciamento de biblioteca pessoal de leitura, desenvolvido em PHP com MySQL. O projeto foi construído utilizando o ambiente XAMPP e seguiu uma abordagem modular com separação entre lógica de negócio e apresentação.

1.1 Especificações Técnicas:

Linguagem Backend: PHP 7.4+

Banco de Dados: MySQL/MariaDB 10.4.32

Servidor Web: Apache (XAMPP)

Frontend: HTML5, CSS3

Arquitetura: MVC simplificado com templates

2. Processo de Desenvolvimento

2.1 Configuração do Ambiente

O primeiro passo consistiu na configuração completa do ambiente de desenvolvimento local:

Instalação e Configuração do XAMPP:

Download e instalação do XAMPP para Windows

Configuração dos módulos Apache e MySQL

Definição da pasta do projeto: C:\xampp\htdocs\LTP3-CRUD-Gerenciador-de-Livros

Estrutura de Diretórios:

LTP3-CRUD-Gerenciador-de-Livros/

```
|— css/
|   |— # style.css
|— templates/
|   |— confirmar_exclusao.php
|   |— footer.php
|   |— form_adicionar.php
|   |— form_editar.php
|   |— header.php
|   |— lista_livros.php
|— adicionar_livro.php
|— conexao.php
|— editar.php
|— excluir.php
|— index.php
|— mudar_status.php
|— teste.php
```

2.2 Modelagem do Banco de Dados

Criação do Banco:

```
sqlCREATE DATABASE gerenciador_livros
```

```
CHARACTER SET utf8mb4
```

```
COLLATE utf8mb4_unicode_ci;
```

Estrutura da Tabela Principal:

```
sqlCREATE TABLE `livros` (
```

```
  `id` INT(11) NOT NULL AUTO_INCREMENT,
```

```
  `titulo` VARCHAR(255) NOT NULL,
```

```
  `autor` VARCHAR(255) NOT NULL,
```

```
  `ano_publicacao` INT(4) DEFAULT NULL,
```

```
  `genero` VARCHAR(100) DEFAULT NULL,
```

```
  `numero_paginas` INT(5) DEFAULT NULL,
```

```
  `status_leitura` TINYINT(1) NOT NULL DEFAULT 0,
```

```
  PRIMARY KEY (`id`)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

Justificativa das Escolhas:

VARCHAR(255): Adequado para títulos e autores, permitindo nomes extensos

TINYINT(1) para status: Permite três estados (0: Não Lido, 1: Lendo, 2: Lido)

utf8mb4: Suporte completo a caracteres especiais e emojis

AUTO_INCREMENT: Garante IDs únicos e sequenciais automaticamente

2.3 Camada de Conexão (conexao.php)

Implementação:

```
<?php

$servidor = "localhost";
$usuario = "root";
$senha = "root";
$banco = "gerenciador_livros";
$porta = 3307;

$conexao = mysqli_connect($servidor, $usuario, $senha, $banco, $porta);
//

if (!$conexao) {
    die("Falha na conexão: " . mysqli_connect_error()); //
}

?>
```

Características:

Centralização da configuração de conexão

Validação imediata de conectividade

Porta customizada (3307) para evitar conflitos

Tratamento de erros com mensagens descritivas

Implementação do CRUD

3. CREATE - Adicionar Livros (Arquivo: adicionar_livro.php)

3.1 Fluxo de Funcionamento:

Validação de Requisição: O script primeiro verifica se a página foi acessada via método POST para garantir que os dados venham do formulário.

Captura de Dados: As informações do livro (título, autor, etc.) são capturadas da superglobal \$_POST.

Validação de Campos: É verificado se os campos essenciais, título e autor, não estão vazios. O script é interrompido com uma mensagem de erro caso estejam.

Preparação da Consulta: Uma instrução INSERT é preparada como um "molde" (prepared statement), usando ? como marcadores de lugar para os dados. Isso previne ataques de SQL Injection.

Verificação da Preparação: O código checa se a preparação da consulta SQL foi bem-sucedida. Se houver um erro de sintaxe no SQL, o script é interrompido.

Vinculação de Parâmetros: As variáveis PHP contendo os dados do livro são vinculadas (bind_param) aos marcadores ? da consulta preparada, especificando o tipo de cada dado (ex: "s" para string, "i" para integer).

Execução: O comando é finalmente executado (execute()) para inserir o novo livro no banco de dados. Uma verificação de erro é feita nesta etapa também.

Finalização e Redirecionamento: A declaração preparada e a conexão com o banco são fechadas para liberar recursos, e o usuário é redirecionado de volta para a página index.php.

3.2 Código Final com Prepared Statements:

```
<?php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

include 'conexao.php';

if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $titulo = $_POST['titulo'];
    $autor = $_POST['autor'];
    $ano_publicacao = $_POST['ano_publicacao'];
    $genero = $_POST['genero'];
    $numero_paginas = $_POST['numero_paginas'];
    $status_leitura = isset($_POST['status_leitura']) ? 1 : 0;

    if (empty($titulo) || empty($autor)) {
        die("Erro: Título e Autor são obrigatórios.");
    }

    $stmt = $conexao->prepare("
        INSERT INTO livros (titulo, autor, ano_publicacao, genero,
numero_paginas, status_leitura)
        VALUES (?, ?, ?, ?, ?, ?)
    ");

    if (!$stmt) {
        die("Erro na preparação da consulta: " . $conexao->error);
    }
}
```

```

        $stmt->bind_param("ssissi", $titulo, $autor, $ano_publicacao,
$genero, $numero_paginas, $status_leitura);

        if (!$stmt->execute()) {
            die("Erro ao adicionar livro: " . $stmt->error);
        }

        $stmt->close();
        mysqli_close($conexao);

        header("Location: index.php");
        exit();
    }
?>

```

3.3 Formulário de Entrada (templates/form_adicionar.php):

Grid responsivo com 2 colunas

Campos validados no cliente (HTML5 required)

Checkbox para status de leitura

Estilização consistente com o sistema

4. READ - Listagem de Livros (Arquivos: index.php + templates/lista_livros.php)

4.1 Lógica de Separação:

A funcionalidade de listagem é dividida em duas responsabilidades claras para um código mais limpo e organizado:

index.php (Controlador): Atua como o "cérebro" da operação. Ele é responsável por incluir a conexão, executar a consulta SELECT para buscar todos os livros no banco de dados e, em seguida, incluir os arquivos de template para montar a página.

templates/lista_livros.php (Visão): Atua como o "mostruário". É um arquivo puramente de apresentação que recebe os dados já consultados (através da variável \$resultado) e os renderiza em formato de tabela HTML, sem conter lógica de acesso ao banco.

4.2 Implementação do index.php: O script principal orquestra a página: conecta-se ao banco, busca os dados e inclui as partes do template em sequência para renderizar a página completa.

```

<?php
require_once 'conexao.php';

$sql = "SELECT * FROM livros ORDER BY id DESC";
$resultado = mysqli_query($conexao, $sql); //

require_once 'templates/header.php';

```

```

require_once 'templates/form_adicionar.php';

require_once 'templates/lista_livros.php';

require_once 'templates/footer.php';

mysqli_close($conexao); //
?>

```

4.3 Renderização Dinâmica em lista_livros.php: O template itera sobre os resultados da consulta. Para cada livro, uma linha da tabela é gerada dinamicamente, com lógica condicional para exibir o status e as ações apropriadas.

A coluna "Status" usa um switch para exibir um texto e ícone diferente para cada um dos três estados possíveis (Lido, Lendo, Não Lido).

A coluna "Ações" contém um botão de troca de status cujo texto e cor mudam dinamicamente, também via switch, para refletir a próxima ação disponível no ciclo.

```

<?php
if ($resultado && mysqli_num_rows($resultado) > 0):
    while ($livro = mysqli_fetch_assoc($resultado)):
?>
<tr>
<td><?php echo $livro['id']; ?></td>
<td><?php echo htmlspecialchars($livro['titulo']); ?></td>
<td><?php echo htmlspecialchars($livro['autor']); ?></td>
<td><?php echo htmlspecialchars($livro['genero']); ?></td>
<td><?php echo $livro['numero_paginas']; ?></td>
<td>
    <?php
        switch ($livro['status_leitura']) {
            case 1:
                echo '<span class="status-lido">✓ Lido</span>';
                break;
            case 2:
                echo '<span class="status-lendo">📖 Lendo</span>';
                break;
            default:
                echo '<span class="status-nao-lido">✗ Não
Lido</span>';
                break;
        }
    ?>
</td>

```



```

<td class="acoes">
    <!-- Botão de trocar status -->
    <a href="mudar_status.php?id=<?php echo $livro['id']; ?>"
        class="btn btn-sm
        <?php
            switch ($livro['status_leitura']) {
                case 1: echo 'btn-danger'; break; // Para a ação
"Marcar como Não Lido" -> Vermelho
                case 2: echo 'btn-success'; break; // Para a ação
"Marcar como Lido" -> Verde
                default: echo 'btn-warning'; break; // Para a ação
"Marcar como Lendo" -> Amarelo
            }
        ?>">
        <?php
            switch ($livro['status_leitura']) {
                case 1: echo 'X Marcar como Não Lido'; break;
                case 2: echo '✓ Marcar como Lido'; break;
                default: echo '🗑 Marcar como Lendo'; break;
            }
        ?>
    </a>

    <!-- Botão de editar -->
    <a href="editar.php?id=<?php echo $livro['id']; ?>" class="btn
btn-sm btn-primary">
        🖋 Editar
    </a>

    <!-- Botão de excluir -->
    <a href="excluir.php?id=<?php echo $livro['id']; ?>" class="btn
btn-sm btn-danger">
        🗑 Excluir
    </a>
</td>
</tr>
<?php
    endwhile;
else:
?>
<tr>
<td colspan="7" style="text-align: center; padding: 30px; color:
#6c757d;">

```

```
Nenhum livro cadastrado ainda.  
</td>  
</tr>  
<?php endif; ?>
```

4.4 Segurança:

Prevenção de XSS: A função htmlspecialchars() é utilizada ao exibir dados vindos do banco de dados (como título, autor e gênero) para prevenir ataques de Cross-Site Scripting (XSS), garantindo que qualquer tag HTML inserida nesses campos seja renderizada como texto inofensivo.

Verificação de Resultados: O código verifica se a consulta retornou resultados (if (\$resultado && mysqli_num_rows(\$resultado) > 0)) antes de tentar iterar sobre eles, evitando erros caso a tabela esteja vazia.

5. UPDATE - Edição de Livros (Arquivos: editar.php e templates/form_editar.php)

5.1 Funcionamento Dual: O arquivo editar.php possui uma lógica de controle que executa ações diferentes com base no método da requisição HTTP:

Requisição GET: Ocorre quando o usuário clica no link "Editar". O script busca os dados do livro específico no banco e os prepara para exibição no formulário.

Requisição POST: Ocorre quando o usuário envia o formulário com as alterações. O script processa os dados, os valida e atualiza o registro no banco de dados.

5.2 Lógica de Busca (GET): Quando a página é acessada sem dados de formulário, o script verifica se um id foi fornecido na URL. Ele usa um prepared statement para buscar o livro correspondente de forma segura. Se o livro não for encontrado ou se nenhum id for fornecido, o usuário é redirecionado para a página inicial.

```
else if (isset($_GET['id'])) {  
    $id = $_GET['id'];  
  
    $stmt = $conexao->prepare("SELECT * FROM livros WHERE id = ?");  
    $stmt->bind_param("i", $id);  
    $stmt->execute();  
    $resultado = $stmt->get_result();  
    $livro = $resultado->fetch_assoc();  
  
    if (!$livro) {  
        header("Location: index.php");  
        exit();  
    }  
} else {  
    header("Location: index.php");  
    exit();  
}
```

5.3 Lógica de Atualização (POST): Ao receber uma requisição POST, o script captura e valida os dados. Em seguida, prepara e executa uma instrução UPDATE para salvar as alterações no banco. Inclui verificação de erro em cada etapa e, ao final, redireciona o usuário para a página principal.

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $id = $_POST['id'];
    $titulo = $_POST['titulo'];
    $autor = $_POST['autor'];
    $ano_publicacao = $_POST['ano_publicacao'];
    $genero = $_POST['genero'];
    $numero_paginas = $_POST['numero_paginas'];
    $status_leitura = isset($_POST['status_leitura']) ? 1 : 0;

    if (empty($titulo) || empty($autor)) {
        die("Erro: Título e Autor são obrigatórios.");
    }

    $stmt = $conexao->prepare("
        UPDATE livros
        SET titulo = ?, autor = ?, ano_publicacao = ?, genero = ?,
            numero_paginas = ?, status_leitura = ?
        WHERE id = ?
    ");

    if (!$stmt) {
        die("Erro na preparação da consulta: " . $conexao->error);
    }

    $stmt->bind_param("ssissii", $titulo, $autor, $ano_publicacao,
$genero,
        $numero_paginas, $status_leitura, $id);

    if ($stmt->execute()) {
        $stmt->close();
        mysqli_close($conexao);
        header("Location: index.php");
        exit();
    } else {
        die("Erro ao atualizar o livro: " . $stmt->error);
    }
}
```

5.4 Formulário de Edição (templates/form_editar.php): A parte visual do formulário é separada em um arquivo de template para melhor organização.

Contém um campo oculto (<input type="hidden">) que armazena e reenvia o id do livro.

Os campos do formulário são pré-preenchidos com os dados do livro, utilizando a sintaxe value="<?php echo \$livro['campo']; ?>".

O checkbox de status de leitura é marcado condicionalmente com base no valor vindo do banco de dados (<?php if (\$livro['status_leitura'] == 1) echo 'checked'; ?>).

6. DELETE - Exclusão de Livros (Arquivos: excluir.php e templates/confirmar_exclusao.php)

6.1 Fluxo com Confirmação: A exclusão é implementada em duas etapas para garantir a segurança e prevenir cliques acidentais:

Requisição GET: Ao clicar no link "Excluir" na listagem, o usuário é direcionado para a página excluir.php. O script busca os dados do livro correspondente ao id passado na URL e exibe uma tela de confirmação.

Requisição POST: Se o usuário clicar no botão "Confirmar Exclusão" na tela de confirmação, um formulário POST é enviado para o mesmo excluir.php, que então executa o comando DELETE no banco de dados e redireciona o usuário para a página principal.

6.2 Implementação do excluir.php: O script excluir.php contém toda a lógica de controle, tratando tanto as requisições GET quanto as POST, e ao final renderiza a página incluindo os templates necessários.

Processamento da Exclusão (POST): Este bloco de código é executado quando o formulário de confirmação é enviado. Ele prepara e executa a instrução DELETE de forma segura.

```
if ($_SERVER["REQUEST_METHOD"] == "POST" && isset($_POST['id'])) {  
    $id = $_POST['id'];  
  
    $stmt = $conexao->prepare("DELETE FROM livros WHERE id = ?");  
  
    if (!$stmt) {  
        die("Erro na preparação da consulta: " . $conexao->error);  
    }  
  
    $stmt->bind_param("i", $id);  
  
    if ($stmt->execute()) {  
        $stmt->close();  
        mysqli_close($conexao);  
        header("Location: index.php");  
        exit();  
    } else {
```

```

        $erro = "Erro ao excluir o livro: " . $stmt->error;
    }
}

```

Busca para Exibição (GET): Este bloco é executado quando a página é carregada inicialmente. Ele busca os dados do livro para que possam ser exibidos na tela de confirmação.

```

else if (isset($_GET['id'])) {
    $id = $_GET['id'];

    $stmt = $conexao->prepare("SELECT * FROM livros WHERE id = ?");
    $stmt->bind_param("i", $id);
    $stmt->execute();
    $resultado = $stmt->get_result();
    $livro = $resultado->fetch_assoc();

    if (!$livro) {
        header("Location: index.php");
        exit();
    }
} else {
    header("Location: index.php");
    exit();
}

```

6.3 Interface de Confirmação (templates/confirmar_exclusao.php): A interface de confirmação é renderizada por um template separado e é projetada para alertar o usuário sobre a ação destrutiva, incluindo:

Destaque visual: Geralmente com uma borda ou fundo em tons de vermelho para indicar perigo.

Exibição dos dados: O título e o autor do livro são claramente exibidos para que o usuário tenha certeza do item que está prestes a apagar.

Botões de Ação: Apresenta dois botões distintos: "Confirmar Exclusão" (vermelho) e "Cancelar" (cinza), que simplesmente retorna à página principal.

7. Funcionalidade Extra: Troca Rápida de Status (Arquivo: mudar_status.php)

7.1 Ciclo de Status: O script implementa um ciclo de três estados específico, projetado para alternar os valores da coluna `status_leitura` na seguinte ordem, utilizando uma estrutura `switch`:

0 (Não Lido) → 2 (Lendo)

2 (Lendo) → 1 (Lido)

1 (Lido) → 0 (Não Lido)

7.2 Implementação:

O script valida o id recebido via GET, busca o status atual do livro no banco, calcula o próximo status no ciclo e, então, executa uma instrução UPDATE para salvar o novo valor. Todas as etapas de interação com o banco de dados são feitas com prepared statements e incluem verificação de erros para garantir a robustez da operação.

```
<?php
require_once 'conexao.php';

if (isset($_GET['id']) && is_numeric($_GET['id'])) {
    $id = intval($_GET['id']);

    // Buscar o status atual
    $stmt = $conexao->prepare("SELECT status_leitura FROM livros WHERE
id = ?");
    if (!$stmt) {
        die("Erro na preparação da consulta: " . $conexao->error);
    }
    $stmt->bind_param("i", $id);
    if (!$stmt->execute()) {
        die("Erro ao executar consulta: " . $stmt->error);
    }

    $resultado = $stmt->get_result();
    $livro = $resultado->fetch_assoc();

    if ($livro) {
        $status_atual = (int)$livro['status_leitura'];

        // Alternar status: 0 → 2 → 1 → 0
        switch ($status_atual) {
            case 0: // Não lido → Lendo
                $novo_status = 2;
                break;
            case 2: // Lendo → Lido
                $novo_status = 1;
                break;
            default: // Lido → Não lido
                $novo_status = 0;
                break;
        }

        // Atualizar no banco
    }
}
```

```

        $stmt_update = $conexao->prepare("UPDATE livros SET
status_leitura = ? WHERE id = ?");
        if (!$stmt_update) {
            die("Erro na preparação do update: " . $conexao->error);
        }
        $stmt_update->bind_param("ii", $novo_status, $id);
        if (!$stmt_update->execute()) {
            die("Erro ao atualizar status: " . $stmt_update->error);
        }
        $stmt_update->close();
    }

    $stmt->close();
    mysqli_close($conexao);

    header("Location: index.php");
    exit();
} else {
    die("Erro: ID inválido ou não fornecido.");
}
?>

```

8. Camada de Apresentação

8.1 Sistema de Templates

A estrutura visual do projeto é modularizada em arquivos de template para separar a lógica PHP do código HTML, facilitando a manutenção e o reuso. Os arquivos principais são o cabeçalho (header.php) e o rodapé (footer.php).

8.2 Arquivo: templates/header.php

Este arquivo inicia a estrutura de todas as páginas HTML. Ele contém o doctype, a tag <head> (com metadados, o título e a folha de estilos externa), e a abertura da tag <body> e de um div container principal.

```

<!DOCTYPE html>
<html lang="pt-br">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Gerenciador de Livros</title>
    <link rel="stylesheet" href="css/style.css">

```

```
</head>
<body>
  <div class="container">
```

8.3 Arquivo: templates/footer.php

Este arquivo é responsável por fechar as tags abertas no header.php, garantindo que todas as páginas tenham uma estrutura HTML válida.

```
</div>
</body>
</html>
```

8.4 Vantagens da Abordagem:

Reuso de código (DRY - Don't Repeat Yourself): O mesmo cabeçalho e rodapé são usados em todas as páginas, evitando duplicação.

Facilita manutenção do layout: Para alterar o título do site ou adicionar um novo arquivo CSS, basta editar o header.php uma única vez.

Separação de responsabilidades: O código PHP foca na lógica de negócio, enquanto os templates focam apenas na apresentação.

CSS externo para melhor performance: O uso de <link rel="stylesheet" href="css/style.css"> permite que o navegador armazene o arquivo de estilos em cache, acelerando o carregamento das páginas subsequentes.

9. Arquivo: css/style.css

9.1 Design "Neumorphic/Solid":

Bordas grossas (4px) para destaque

Sombras marcadas (8px 8px 15px)

Fundo cinza claro (#ECF0F1)

Containers brancos com transparência

9.2 Sistema de Cores Consistente:

Azul (#4ca7d2): Ações primárias

Verde (#60d24c): Sucesso/Lido

Amarelo (#ffa500): Lendo/Aviso

Vermelho (#f43c23): Excluir/Não Lido

Cinza escuro (#2C3E50): Títulos e bordas

9.3 Grid Responsivo: O formulário de entrada utiliza CSS Grid para se adaptar a diferentes tamanhos de tela. Em telas largas, os campos se organizam em colunas; em telas menores, empilham-se em uma única coluna.

```
.form-grid {
  display: grid;
```



```
grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
gap: 15px;
margin-bottom: 20px;
```

9.4 Tabela Estilizada:

Cabeçalho escuro (#2C3E50)

Linhas com hover (#ECF0F1)

Bordas arredondadas

Overflow hidden para bordas limpas

9.5 Responsividade:

```
@media (max-width: 768px) {
  .container {
    padding: 15px;
    margin: 10px;
    border-width: 3px;
  }

  .form-grid {
    grid-template-columns: 1fr;
  }

  table {
    font-size: 12px;
    border-width: 2px;
  }

  th, td {
    padding: 8px;
  }

  .acoes a {
    display: block;
    margin: 5px 0;
  }

  h1 {
    font-size: 20px;
  }
}
```

Desafios e Soluções

10. Conflito de Porta MySQL

10.1 Problema

Durante a inicialização do XAMPP, o MySQL apresentou erro indicando que a porta padrão 3306 estava em uso por outro serviço.

Port 3306 in use by "Unable to open process"

Causa Raiz: Outra instalação do MySQL/MariaDB estava configurada como serviço do Windows e iniciava automaticamente.

10.2 Solução Implementada

Identificação do Processo:

bash netstat -ano | find "3306"

Desativação do Serviço Conflitante:

Acesso ao services.msc

Localização do serviço MySQL80/MariaDB

Alteração do tipo de inicialização para "Manual"

10.3 Reconfiguração do XAMPP:

Edição do arquivo C:\xampp\mysql\bin\my.ini

Alteração das linhas:

[client]

port=3307

[mysqld]

port=3307

Atualização do Código:

php \$conexao = mysqli_connect(\$servidor, \$usuario, \$senha, \$banco, 3307);

10.4 Lições Aprendidas:

Sempre documentar configurações não-padrão

Verificar conflitos de porta antes da instalação

Manter múltiplos ambientes isolados

11. Erro de Autenticação MySQL

11.1 Problema:

Após várias tentativas de reset de senha, o erro persistia:

Access denied for user 'root'@'localhost' (using password: NO)

11.2 Tentativas Iniciais (Sem Sucesso):

Uso de comandos antigos (mysql_resetroot.bat inexistente na versão atual)

ALTER USER não funcionando em modo --skip-grant-tables

Comandos UPDATE diretos na tabela mysql.user bloqueados

11.3 Solução Final:

Inicialização em Modo Seguro:

```
bash mysqld --skip-grant-tables
```

Conexão e Reativação de Permissões:

```
bash mysql -u root
```

```
sql FLUSH PRIVILEGES;
```

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'root';
```

```
exit;
```

Atualização da Configuração do phpMyAdmin:

```
php // Arquivo: C:\xampp\phpMyAdmin\config.inc.php
```

```
$cfg['Servers'][$i]['password'] = 'root';
```

```
$cfg['Servers'][$i]['AllowNoPassword'] = false;
```

11.4 Descoberta Importante:

A sequência correta era executar FLUSH PRIVILEGES; antes de tentar alterar a senha, para "acordar" o sistema de permissões mesmo em modo seguro.

11.5 Impacto:

Este desafio consumiu significativo tempo de desenvolvimento, mas proporcionou compreensão profunda sobre:

Funcionamento interno do sistema de autenticação MySQL

Diferenças entre versões antigas e modernas do MariaDB

Importância de documentação atualizada

12. Corrupção da Tabela do Banco de Dados

12.1 Problema:

Após múltiplas tentativas de criação/recriação, a tabela livros aparecia no phpMyAdmin mas retornava:

```
Table 'gerenciador_livros.livros' doesn't exist in engine
```

12.2 Diagnóstico:

Referência da tabela existia no dicionário do MySQL

Arquivo físico .ibd estava ausente ou corrompido

Causado por travamentos durante operações de DDL

12.3 Solução:

Parada Completa do MySQL:

XAMPP Control Panel → Stop MySQL

Limpeza Manual:

C:\xampp\mysql\data\gerenciador_livros\

- Deletar: livros.ibd

- Deletar: livros.frm (se existir)

Reinício e Recriação:

```
bash # Iniciar MySQL
```

```
# Acessar phpMyAdmin
```

```
sql DROP TABLE IF EXISTS livros;
```

```
CREATE TABLE livros (...);
```

Recursos e Ferramentas

13. XAMPP (v8.2.12)

13.1 Componentes Utilizados:

Apache 2.4.58: Servidor web

MariaDB 10.4.32: Sistema de banco de dados

PHP 8.2.12: Linguagem de programação backend

phpMyAdmin 5.2.1: Interface gráfica para MySQL

13.2 Vantagens:

Ambiente integrado e pré-configurado

Eliminação de complexidade de configuração individual

Ideal para desenvolvimento local

Painel de controle unificado

13.3 Uso no Projeto:

Desenvolvimento e testes locais

Administração do banco via phpMyAdmin

Monitoramento de logs (Apache: error.log, MySQL: mysql_error.log)

14. Visual Studio Code

14.1 PHP Intelephense:

Autocompletar inteligente

Validação de sintaxe em tempo real

Navegação por definições

14.2 MySQL (Weijan Chen):

Conexão direta com banco de dados

Execução de queries SQL

Visualização de estruturas

14.3 HTML CSS Support:

Autocompletar para classes CSS

Validação de HTML

14.4 Prettier - Code Formatter:

Formatação automática de código

Consistência de estilo

15. Bootstrap Classes (Inline)

15.1 Decisão de Design:

Uso de classes do Bootstrap apenas para cores de botões, mantendo CSS customizado para layout.

15.2 Classes Utilizadas:

```
css/* Cores de ação */  
.btn-primary { background-color: #3498db; } /* Azul */  
.btn-success { background-color: #60d24c; } /* Verde */  
.btn-warning { background-color: #ffa500; } /* Amarelo */  
.btn-danger { background-color: #f43c23; } /* Vermelho */  
.btn-secondary { background-color: #95a5a6; } /* Cinza */
```

15.3 Justificativa:

Convenção amplamente conhecida (semântica de cores)
Facilita manutenção futura
Reduz necessidade de classes customizadas
Mantém coesão visual

16. mysqli (PHP Extension)

16.1 Escolha Tecnológica:

Uso de mysqli ao invés de PDO ou mysql (deprecated).

16.2 Vantagens do mysqli:

Suporte a prepared statements
Orientado a objetos e procedimental (flexibilidade)
Melhor performance com MySQL/MariaDB
Funções específicas otimizadas

16.3 Padrão de Uso:

```
Estilo procedimental (usado no projeto)  
$conexao = mysqli_connect($servidor, $usuario, $senha, $banco, $porta);  
$stmt = mysqli_prepare($conexao, $sql);  
mysqli_stmt_bind_param($stmt, "s", $valor);  
mysqli_stmt_execute($stmt);
```

// Também suporta OOP

```
$conexao = new mysqli($servidor, $usuario, $senha, $banco, $porta);  
$stmt = $conexao->prepare($sql);
```

17. Chrome DevTools

17.1 Console JavaScript:

Debug de erros de frontend
Teste de interações

17.2 Network Tab:

Monitoramento de requisições POST/GET

Verificação de redirecionamentos
Análise de tempos de resposta

17.3 Elements/Inspector:
Ajuste fino de CSS
Verificação de estrutura HTML gerada
Teste de responsividade

18. Recursos Consultados

18.1 php.net:
Exemplos de prepared statements
Documentação de mysqli_* functions

18.2 dev.mysql.com:
Sintaxe SQL específica do MariaDB
Otimização de queries

18.3 MDN Web Docs:
HTML5 semântico
CSS Grid e Flexbox
Boas práticas de acessibilidade

18.4 Stack Overflow:
Padrões de código recomendados
Troubleshooting de erros

18.5 W3Schools:
Tutoriais e exemplos práticos de CSS
Referência rápida para propriedades display e visibility

18.6 HTML Color Codes:
Ferramenta para seleção de cores
Gerador de códigos de cor (Hex, RGB)
Paletas de cores para inspiração de design

19. Técnicas Implementadas

19.1 Exibição de Erros:

```
ini_set('display_errors', 1);  
ini_set('display_startup_errors', 1);  
error_reporting(E_ALL);
```